

Algoritmos de Ordenação





Algoritmo de Ordenação

- Algoritmos de ordenação recebem dados desorganizados, ordenando os mesmos segundo algum critério e facilitando assim seu uso.
- Tais algoritmos são relevantes na redução de complexidade em outros problemas como, por exemplo, no problema computacional de busca.



Algoritmo de Ordenação

- Alguns aspectos devem ser considerados na aplicação de algoritmos de ordenação:
  - Tamanho do conjunto de dados a ser ordenado.
  - Memória disponível para armazenamento e manipulação desses dados.
- Alguns algoritmos demandam maior espaço em memória durante sua execução, enquanto outros economizam memória ao custo de serem mais lentos.



Algoritmo de Ordenação

- Algoritmos de ordenação fornecem soluções para o problema computacional de ordenação.
- Problema Computacional de Ordenação
  - Input: Uma sequência de n números:$\{a_1\ a_2, a_3, ..., a_n\}$
  - Output: Uma reordenação da sequência em $\{a'_1, a'_2, a'_3 ... a'_n\}$ tal que $a'_1 \leq a'_2 \leq a'_3 \leq ... \leq a'_n$



Algoritmo de Ordenação Bubble Sort

- Algoritmo simples de ordenação e fácil de implementar que executa comparações entre todos os elementos.
- Recebeu esse nome por conta dos valores a serem ordenados se comportarem como bolhas em um recipiente que, de acordo com seu tamanho, ajustam-se a determinado patamar.





Algoritmo de Ordenação Bubble Sort

- Desvantagem: apresenta um desempenho ruim, quando comparado aos outros algoritmos de ordenação, ocasionado pelas várias comparações e trocas realizadas.
- Vantagens: fácil de compreender e implementar



Alg bubbleSortOtimizado(L[1,...,n],n):

```
Para i= 0 até n faça
    troca = F
    Para j= i+1 até n faça
        Se L[i] > L[j] então
            L[j],L[i] = L[i],L[j]
            troca = V
    Se( troca==F) então
            Parar
retorne L
```

i

0	1	2	3	4	5
12	81	44	23	67	90

j

0	1	2	3	4	5
12	81	44	23	67	90

j

0	1	2	3	4	5
12	44	81	23	67	90

j

0	1	2	3	4	5
12	44	81	23	67	90

j

0	1	2	3	4	5
12	23	81	44	67	90

j

0	1	2	3	4	5
12	23	81	44	67	90

j

0	1	2	3	4	5
12	23	81	44	67	90



```
Alg bubbleSortOtimizado(L[1,...,n],n):
  Para i= 0 até n faça
      troca = F
      Para j= i+1 até n faça
          Se L[i] > L[j] então
              L[j],L[i] = L[i],L[j]
              troca = V
      Se( troca==F) então
              Parar
  retorne L
```

j

0	1	2	3	4	5
12	23	81	44	67	90

0	1	2	3	4	5
12	23	44	81	67	90

0	1	2	3	4	5
12	23	44	81	67	90

j

0	1	2	3	4	5
12	23	44	67	81	90

0	1	2	3	4	5
12	23	44	67	81	90

j

0	1	2	3	4	5
12	23	44	67	81	90

j

0	1	2	3	4	5
12	23	44	67	81	90



Algoritmo de Ordenação Bubble Sort

Análise da complexidade de tempo do algoritmo:

- **Melhor caso**: ocorre quando o conjunto de dados está ordenado.
  - As operações do laço interno serão executadas uma única vez para i = 1 no caso da versão melhorada do algoritmo.
  - A comparação será realizada n - 1 vezes. Portanto, a complexidade do Bubble Sort no melhor caso é O(n)



Algoritmo de Ordenação Bubble Sort

Análise da complexidade de tempo do algoritmo:

- **Pior caso**: ocorre quando o vetor está em ordem decrescente.
    - A comparação será sempre verdadeira, executando a troca em todas as iterações.



Algoritmo de Ordenação Bubble Sort

Análise da complexidade de tempo do algoritmo:

- **Pior caso**: ocorre quando o vetor está em ordem decrescente.
  - O total de iterações do laço aninhado será n-1 e considerando os dois laços, t$\sum_{i=1}^{n-1}\left(\sum_{j=1}^{n-i}1\right)$s: ⟹ $\sum_{i=1}^{n-1}(n-i)$



Algoritmo de Ordenação Bubble Sort

Análise da complexidade de tempo do algoritmo:

- **Pior caso**: ocorre quando o vetor está em ordem decrescente.

$$\sum_{i=1}^{n-1}(n-i) = n(n-1) - \frac{n(n-1)}{2}$$

$$= \frac{1}{2}n(n-1) = \frac{1}{2}n^2 - \frac{1}{2}n$$

$$= O(n^2)$$



Algoritmo de Ordenação Insertion Sort

- Algoritmos de ordenação fornecem soluções para o problema computacional de ordenação.
- Problema Computacional de Ordenação
  - Input: Uma sequência de n números:$\{a_1\ a_2, a_3, ..., a_n\}$
  - Output: Uma reordenação da sequência em $\{a'_1, a'_2, a'_3 ... a'_n\}$ tal que $a'_1 \leq a'_2 \leq a'_3 \leq ... \leq a'_n$



Algoritmo de Ordenação Insertion Sort

- Executa a comparação com os elementos das posições anteriores a partir do segundo elemento do vetor, **buscando executar inserções na posição correta a cada iteração**.



Algoritmo de Ordenação Insertion Sort

- Divide a estrutura que armazena os dados em duas partições:
  - Uma partição ordenada à esquerda
  - Uma partição não ordenada à direita.
  - A partição à esquerda terá inicialmente um único elemento

  .



Algoritmo de Ordenação Insertion Sort

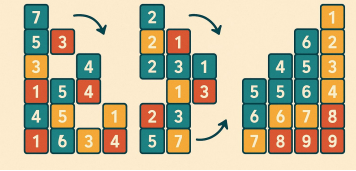
- Logo, a inserção é executada em duas etapas:
  - Etapa 1: encontra-se a posição de inserção.
  - Etapa 2: libera a posição de inserção deslocando os dados da esquerda para a direita.




```
INSERTION-SORT (L, n)
Para j = 1 até n faça
    key = L[ j]
    i = j –1
    Enquanto (i > 0) E  L[i] >key faça
      L[i+1] =  L[i]
       i = i –1
    L[i+1] = key
retorne L
```

Lista L

1	2	3	4	5	6	7	8	9	10
81	12	44	23	67	90	15	9	22	50



```
INSERTION-SORT (L, n)
Para j = 1 até n faça
    key = L[ j]
    i = j –1
    Enquanto (i > 0) E  L[i] >key faça
        L[i+1] =  L[i]
        i = i –1
    L[i+1] = key
retorne L
```

key 12

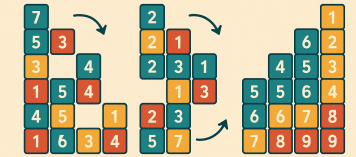
i	j	1	2	3	4	5	6	7	8	9	10
		81	12	44	23	67	90	15	9	22	50

key 12

i	j	0	1	2	3	4	5	6	7	8	9	10
			81	81	44	23	67	90	15	9	22	50

key 12

i	j	0	1	2	3	4	5	6	7	8	9	10
			12	81	44	23	67	90	15	9	22	50

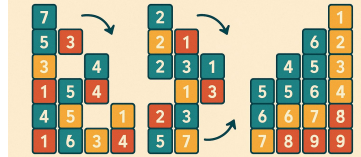


```
INSERTION-SORT (L, n)
Para j = 1 até n faça
    key = L[ j]
    i = j –1
    Enquanto (i > 0) E  L[i] >key faça
        L[i+1] =  L[i]
        i = i –1
    L[i+1] = key
retorne L
```

		i	j							Key←44
0	1	2	3	4	5	6	7	8	9	10
	12	81	44	23	67	90	15	9	22	50

		i	j							Key←44
0	1	2	3	4	5	6	7	8	9	10
	12	81	81	23	67	90	15	9	22	50

		i	j							Key←44
0	1	2	3	4	5	6	7	8	9	10
	12	44	81	23	67	90	15	9	22	50

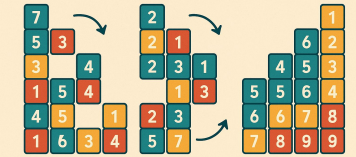


```
INSERTION-SORT (L, n)
Para j = 1 até n faça
    key = L[ j]
    i = j –1
    Enquanto (i > 0) E  L[i] >key faça
        L[i+1] =  L[i]
        i = i –1
    L[i+1] = key
retorne L
```

			i	j						Key←23
0	1	2	3	4	5	6	7	8	9	10
	12	44	81	23	67	90	15	9	22	50

				i	j					Key←23
0	1	2	3	4	5	6	7	8	9	10
	12	44	81	81	67	90	15	9	22	50

				i	j					Key←23
0	1	2	3	4	5	6	7	8	9	10
	12	44	44	81	67	90	15	9	22	50



```
INSERTION-SORT (L, n)
Para j = 1 até n faça
    key = L[ j]
    i = j –1
    Enquanto (i > 0) E  L[i] >key faça
        L[i+1] =  L[i]
        i = i –1
    L[i+1] = key
retorne L
```

	i			j						Key←23
0	1	2	3	4	5	6	7	8	9	10
	12	23	44	81	67	90	15	9	22	50

				i	j					Key←67
0	1	2	3	4	5	6	7	8	9	10
	12	23	44	81	67	90	15	9	22	50

			i		j					Key←67
0	1	2	3	4	5	6	7	8	9	10
	12	23	44	67	81	90	15	9	22	50



Algoritmo de Ordenação Insertion Sort

```
INSERTION-SORT (L, n)
Para j = 1 até n faça
    key = L[ j]
    i = j –1
    Enquanto (i > 0) E  L[i] >key faça
      L[i+1] =  L[i]
       i = i –1
    L[i+1] = key
retorne L
```

- Desvantagem: lento para ordenar vetores de dimensão alta devido à necessidade de **reposicionamento dos dados**.



Algoritmo de Ordenação Insertion Sort

```
INSERTION-SORT (L, n)
Para j = 1 até n faça
    key = L[ j]
    i = j –1
    Enquanto (i > 0) E  L[i] >key faça
      L[i+1] =  L[i]
       i = i –1
    L[i+1] = key
retorne L
```

- Vantagens: executa poucas comparações e quase nenhuma troca para vetores com algum tipo de ordenação prévia.



Complexidade do Insertion Sort

```
INSERTION-SORT (L, n)
1.  Para j = 1 até n faça
2.      key = L[ j]
3.      i = j –1
4.      Enquanto (i > 0) E  L[i] >key faça
5.          L[i+1] =  L[i]
6.          i = i –1
7.      L[i+1] = key
8.  retorne L
```

- Complexidade de tempo avaliada pela contagem do número de instruções básicas executadas para uma entrada de tamanho n.
- Seja c_m um valor constante que indica o custo em termos de tempo para cada execução das instruções na linha m .
- Seja t_j a quantidade de vezes que as condições dentro do comando de repetição while foram verificadas na linha 4 para cada valor de j atribuído na linha 1.



Complexidade do Insertion Sort

INSERTION-SORT (L, n)	Custo	Número de Execuções
1. Para j = 1 até n faça	c1	n
2. key = L[j]	c2	n-1
3. i = j –1	c3	n-1
4. Enquanto (i > 0) E L[i] >key faça	c4	$\sum_{j=2}^{n} t_j$
5. L[i+1] = L[i]	c5	$\sum_{j=2}^{n} (t_j - 1)$
6. i = i –1	c6	$\sum_{j=2}^{n} (t_j - 1)$
7. L[i+1] = key	c7	n-1
8. retorne L		



Complexidade do Insertion Sort

INSERTION-SORT (L, n)

1. Para j = 1 até n faça
2. key = L[j]
3. i = j –1
4. Enquanto (i > 0) E L[i] >key faça
5. L[i+1] = L[i]
6. i = i –1
7. L[i+1] = key
8. retorne L

- Para cada iteração do laço externo dada pelo valor de j, t_j é o número de vezes que a condição do enquanto na linha 4 é testada.
- Se, para um certo j, o algoritmo precisou deslocar s_j elementos para a direita, então $t_j = s_j + 1$ (porque há um teste extra — o que falha — após o último deslocamento.
- c5 e c6 contam o número de deslocamentos (executam só quando a condição é verdadeira), enquanto c4 conta comparações, incluindo o teste final que dá falso.



Complexidade do Insertion Sort

- Exemplo: Lista: [5,2,4,6]
  - j=2: key=2
    - Comparamos (5>2) → verdadeiro (desloca 5), depois comparamos novamente e para com [2,5,4,6], temos t_2=2
    - Deslocamentos s_2=1 ⇒ t_2=s_2+1=2 ⇒ s_2=t_2-1
  - j=3: key=4,
    - Comparamos (5>4) → verdadeiro (desloca 5), depois comparamos (2>4) → falso e para com [2,4,5,6].
    - Deslocamentos (s_3=1) ⇒ (t_3=2).



Complexidade do Insertion Sort

- Exemplo: Lista: [5,2,4,6]
  - j=4: key=6
    - Comparamos (5>6) → falso e para.
    - Deslocamentos $(s_4=0) \Rightarrow (t_4=1)$.
- Somando:

  c4: $(t_2+t_3+t_4=2+2+1=5)$ (5 testes da condição)

  c5: $(t_2-1)+(t_3-1)+(t_4-1)=1+1+0=2$ (2 deslocamentos)

  c6: igual a c5 = 2 decrementos.



Complexidade do Insertion Sort

INSERTION-SORT (L, n)	Custo	Número de Execuções
1. Para j = 1 até n faça	c1	n
2. key = L[j]	c2	n-1
3. i = j –1	c3	n-1
4. Enquanto (i > 0) E L[i] >key faça	c4	$\sum_{j=2}^{n} t_j$
5. L[i+1] = L[i]	c5	$\sum_{j=2}^{n} (t_j - 1)$
6. i = i –1	c6	$\sum_{j=2}^{n} (t_j - 1)$
7. L[i+1] = key	c7	n-1
8. retorne L		



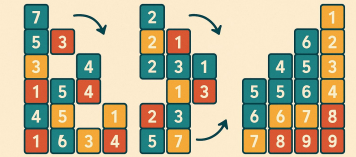
Complexidade do Insertion Sort

INSERTION-SORT (L, n)

1. Para j = 1 até n faça
2. key = L[j]
3. i = j –1
4. Enquanto (i > 0) E L[i] >key faça
5. L[i+1] = L[i]
6. i = i –1
7. L[i+1] = key
8. retorne L

Tempo total de execução para entrada de tamanho n:

$$T(n) = c_1 n + c2(n-1) + c3(n-1) + \\ + c4\sum_{j=2}^{n} t_j + c5\sum_{j=2}^{n}(t_j - 1) + c6\sum_{j=2}^{n}(t_j - 1) + \\ c7(n-1)$$



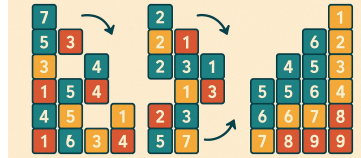
- Tempo total de execução para o melhor caso - Vetor já ordenado
  - $T_j = 1$ pois a condição na linha 4 nunca será satisfeita.

$$T(n) = c_1 n + c2(n-1) + c3(n-1) + \\ + c4 \sum_{j=2}^{n} t_j + c5 \sum_{j=2}^{n} (t_j - 1) + c6 \sum_{j=2}^{n} (t_j - 1) + \\ c7(n-1)$$

$$\begin{cases} t_j = 1 \\ \sum_{j=2}^{n} t_j = n - 1 \\ T(n) = c_1 n + c_2(n-1) + c_3(n-1) + c_4(n-1) + c_7(n-1) \\ T(n) = (c_1 + c_2 + c_3 + c_4 + c_7)n - (c_2 + c_3 + c_4 + c + 7) \\ T(n) = an + b \end{cases}$$



- Tempo total de execução para o pior caso - Vetor ordenado em ordem oposta ao ordenamento proposto.



INSERTION-SORT LA, n)

...

```
L[i+1] =L[i]
i = i –1
```

...

	0	1	2	3	4	5	6	7	8	9	10
		42	53	74	81	37	30	22	15	9	1
					i	j	Key←37				

	0	1	2	3	4	5	6	7	8	9	10
		42	53	53	74	81	30	22	15	9	1
			i			j	Key←37				

	0	1	2	3	4	5	6	7	8	9	10
		42	53	74	81	81	30	22	15	9	1
				i		j	Key←37				

	0	1	2	3	4	5	6	7	8	9	10
		42	42	53	74	81	30	22	15	9	1
		i				j	Key←37				

	0	1	2	3	4	5	6	7	8	9	10
		42	53	74	74	81	30	22	15	9	1
				i		j	Key←37				

	0	1	2	3	4	5	6	7	8	9	10
		37	42	53	74	81	30	22	15	9	1
		i				j	A[i+1] =key				

- **Tempo total de execução para o pior caso - Vetor ordenado em ordem oposta ao ordenamento proposto.**

$$T(n) = c_1 n + c2(n-1) + c3(n-1) + \\ + c4\sum_{j=2}^{n} t_j + c5\sum_{j=2}^{n}(t_j - 1) + c6\sum_{j=2}^{n}(t_j - 1) + \\ c7(n-1)$$

$$t_j = j$$

$$\sum_{j=2}^{n} t_j = \sum_{j=2}^{n} j = 2 + 3 + 4 + 5 + 6 \ldots n = \frac{n(n+1)}{2}$$ -1

$$\sum_{j=2}^{n}(t_j - 1) = \sum_{j=2}^{n}(j-1) = \frac{n(n-1)}{2}$$

$$T(n) = c_1 n + c_2(n-1) + c_3(n-1) + \\ + c_4\frac{n(n+1))}{2} + +c_5\frac{n(n-1)}{2} + c_6\frac{n(n-1)}{2} + c_7(n-1)$$

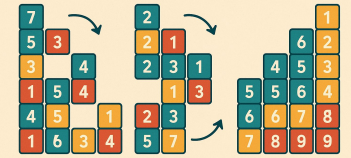
$$T(n) = (\frac{c_4 + c_5 + c_6}{2})n^2 + [c_1 + c_2 + c_3 + c_7 + \frac{(c_4 - c_5 - c_6)}{2}]n \\ - (c_2 + c_3 + c_7)$$



- Vamos analisar o caso médio do algoritmo insertion sort.
- Para isso, considere que a ordenação será aplicada sobre uma lista com valores aleatoriamente gerados.
- Nesse caso, avaliamos o tempo necessário para encontrar o subarranjo L[1 .. j – 1] onde a inserção ocorrerá.



- Numa lista com valores aleatoriamente gerado, podemos supor que metade dos elementos de L[1 .. j – 1] é menor em média que o valor a ser inserido.
- Logo, comparamos em média com metade dos elementos do subarranjo L[1 .. j – 1].
  - tj = j/2.



- tj = j/2 não impacta em T(n)
- O tempo de execução no caso médio continua sendo uma função quadrática do tamanho da entrada.
- O caso médio é quase tão custoso quanto o pior caso.

$$T(n) = (\frac{c_4 + c_5 + c_6}{2})n^2 + [c_1 + c_2 + c_3 + c_7 + \frac{(c_4 - c_5 - c_6)}{2}]n$$

$$- (c_2 + c_3 + c_7)$$

$T(n)=an^2+bn+c$
a,b e c constantes



Algoritmo de Ordenação Selection Sort

- Algoritmos de ordenação fornecem soluções para o problema computacional de ordenação.
- Problema Computacional de Ordenação
  - Input: Uma sequência de n números:$\{a_1\ a_2, a_3, ..., a_n\}$
  - Output: Uma reordenação da sequência em $\{a'_1, a'_2, a'_3 ... a'_n\}$ tal que $a'_1 \leq a'_2 \leq a'_3 \leq ... \leq a'_n$



Algoritmo de Ordenação Selection Sort

- Também compara **todos os elementos utilizando o aninhamento** de dois laços.
- Diferente do bubble sort, executa uma única troca durante as comparações

```
algSelectionSort(L,n)
    Para i=0 até n-1 faça
        min = i
        Para j = i+1 até n faça
            Se(L[min]>L[j]) então
                min = j
        temp = L[i]
        L[i] = L[min]
        L[min] = temp
```



Algoritmo de Ordenação Selection Sort

- O algoritmo compara um elemento com todos os seguintes, armazenando a posição do menor elemento encontrado nas comparações.
- Ao terminar as comparações, realiza a troca do elemento avaliado com o menor encontrado.

```
algSelectionSort(L,n)
    Para i=0 até n-1 faça
        min = i
        Para j = i+1 até n faça
            Se(L[min]>L[j]) então
                min = j
        temp = L[i]
        L[i] = L[min]
        L[min] = temp
```



Algoritmo de Ordenação Selection Sort

- Desvantagem: lento para ordenar vetores de dimensão alta como o Bubble sort.
- Vantagens: executa uma única operação de troca para o item avaliado em cada iteração.

```
algSelectionSort(L,n)
    Para i=0 até n-1 faça
        min = i
        Para j = i+1 até n faça
            Se(L[min]>L[j]) então
                min = j
        temp = L[i]
        L[i] = L[min]
        L[min] = temp
```



	id=i	j			
0	1	2	3	4	5
12	81	44	23	67	90

	i		id		j
0	1	2	3	4	5
12	81	44	23	67	90

	i	id	j		
0	1	2	3	4	5
12	81	44	23	67	90

	i		id	j	
0	1	2	3	4	5
12	23	44	81	67	90

	i		id	j	
0	1	2	3	4	5
12	81	44	23	67	90

		i=id	j		
0	1	2	3	4	5
12	23	81	44	67	90

		i	id	j	
0	1	2	3	4	5
12	23	81	44	67	90

algSelectionSort(L,n)

```
Para i=0 até n-1 faça
    min = i
    Para j = i+1 até n faça
        Se(L[min]>L[j]) então
            min = j
    temp = L[i]
    L[i] = L[min]
    L[min] = temp
```



Algoritmo de Ordenação Selection Sort

```
algSelectionSort(L,n)                                   O(n)
    Para i=0 até n-1 faça
        min = i                                                  O(n)
        Para j = i+1 até n faça
            Se(L[min]>L[j]) então
                min = j
        temp = L[i]
        L[i] = L[min]
        L[min] = temp
```



Analisando Algoritmos

- Analisando a complexidade de tempo, considerando o total de comparações no Selection Sort , temos:

$$\sum_{i=1}^{n-1}\sum_{j=i+1}^{n} 1 = \sum_{i=1}^{n-1}(n-i) = n(n-1) - \frac{n(n-1)}{2}$$

$$= \frac{n(n-1)}{2} \quad = \frac{n(n-1)}{2} \quad = \frac{n^2}{2} - \frac{n}{2}$$

$$=\Theta(n^2)$$



Algoritmo de Ordenação Selection Sort

Bubble Sort vs Insertion Sort vs Selection Sort

- Todos são $O(n^2)$ considerando a complexidade de tempo no pior caso.
- Bubble Sort e Insertion sort (otimizado) são O(n) no melhor caso de O(n), enquanto Selection Sort permanece $O(n^2)$.
- Logo, Selection Sort é $\Theta(n^2)$



Algoritmo de Ordenação Selection Sort

Bubble Sort vs Insertion Sort vs Selection Sort

- Considerando conjunto de dados com dimensão reduzida:
  - Insertion Sort costuma superar Bubble Sort e Selection Sort
  - Bubble Sort e Selection Sort podem apresentar desempenho superior ao Insertion Sort para conjuntos de dados parcialmente ordenado, ou conjuntos de dados maiores.



Algoritmo de Ordenação Selection Sort

- Todos apresentam complexidade de espaço O(1).
- Bubble Sort e Insertion Sort são chamados algoritmos de classificação estáveis por manterem a ordem relativa de elementos iguais na estrutura classificada, enquanto Selection Sort não é estável.
- As características específicas do problema computacional de ordenação são determinantes para a escolha do melhor algoritmo, todos apresentam vantagens e desvantagens



Algoritmo Estáveis

- Chama-se estável um algoritmo de ordenação que mantém a ordem relativa de elementos iguais.
- Formalmente, atende a seguinte propriedade:
  - A é um algoritmo de ordenação estável se, e somente se, se dois elementos a e b têm a mesma chave com a precedendo b antes da ordenação, então a continuará precedendo b após a ordenação.



Algoritmo Estáveis

L=	1	2	3	2	4

L=	1	2a	3	2b	4

Estável

L=	1	2a	2b	3	4

Instável

L=	1	2b	2a	3	4



Algoritmo Estáveis

- Tais algoritmos são relevantes em aplicações como bancos de dados, onde a ordem original dos elementos deve ser preservada.
- Os algoritmos abaixo são estáveis:
  - Bubble Sort
  - Merge Sort
  - Insertion Sort



Algoritmos In-place

- Abaixo algumas definições para o conceito de *in-place algorithms*.
  - Não demanda espaço extra para armazenamento, retornando o resultado usando a mesma memória alocada aos dados
    - Logo, é executados sobre a entrada, transformando tal entrada no mesmo espaço de memória - ‘in-place’.
    - Um espaço extra e constante alocado a variáveis pode ser utilizado.



Algoritmos In-place

- Abaixo algumas definições para o conceito de *in-place algorithms*.
  - Não demanda espaço extra na manipulação dos dados de entrada, mas pode alocar uma quantidade extra de memória para sua operação.
    - O espaço extra alocado geralmente é O(log n), mas ordem de crescimento O(n) pode ser tolerada.



Algoritmos In-place

- Abaixo algumas definições para o conceito de *in-place algorithms*.
  - A memória extra alocada é independente do tamanho da instância do problema de ordenação;



Algoritmos In-place

- Bubble Sort é um algoritmo in-place.
  - Realiza repetidas trocas entre elementos adjacentes, se estiverem na ordem errada, até que nenhuma troca seja necessária.
  - Para isso, utiliza apena variável temporária para a troca com uma complexidade de espaço O(1).



Algoritmos In-place

- Insertion Sort é um algoritmo in-place
  - Ordena os elementos mantendo uma partição ordenada, selecionando o próximo elemento da partição não ordenada a cada iteração.
  - Para isso, armazena o elemento atual numa variável (espaço extra demandado) independente do tamanho da instância do problema de ordenação.



Algoritmos In-place

- Selection Sort é um algoritmo in-place
  - Na partição não ordenada, busca a cada iteração o maior ou menor elemento, trocando com o primeiro elemento não ordenado.
  - Para isso, também utiliza variáveis para armazenar o menor ou maior elemento e executar trocas com uma complexidade de espaço O(1).



Complexidade de tempo

- **Complexidade do melhor caso**: lista com dados já está ordenada.
  - Selection sort é $O(n^2)$
    - Precisa percorrer toda a lista a ser ordenada para encontrar o elemento mínimo a cada iteração do laço externo.
  - Bubble sort é O(n)
    - Consegue parar o procedimento após uma única passagem por todos os elementos da lista já que condição do laço interno não é satisfeita.
  - Insertion sort é O(n)
    - Nenhuma troca é realizada já que os elementos estão ordenados.



Complexidade de tempo

- **Complexidade do pior caso**: lista em ordem inversa.
  - Bubble sort é $O(n^2)$
    - A complexidade do pior caso é $O(N^2)$ onde todas as comparações na partição não ordenada precisam ser executadas..
  - Insertion sort é $O(n^2)$
    - Precisa percorrer toda partição ordenada para encontrar a posição correta do elemento.
  - Selection sort é $O(n^2)$
    - Precisa percorrer toda a partição não ordenada para encontrar o menor ou maior elemento a cada iteração.



Complexidade de tempo

- **Complexidade do caso médio**: lista de dados aleatoriamente gerados
  - Todos os três algoritmos apresentam complexidade $O(n^2)$
  - Bubble and Insertion sort realizam trocas entre elementos adjacente, realizando no máximo n-1 trocas adjacentes. Todavia, isso será realizado em torno de n/4 iterações resultando em $O(n^2)$.
  - Selection sort executa uma única troca não adjacente, também reduzindo para n-1 troca. Porém, precisamos percorrer toda partição não ordenada resultando em $O(n^2)$.



Algoritmo de Ordenação Merge Sort

- Merge Sort ou ordenação por mistura utiliza a estratégia de divisão e conquista na ordenação dos valores.



Algoritmo de Ordenação Merge Sort

- Paradigma de divisão e conquista
  - Estratégia que divide o problema original em subproblemas semelhantes e menores.
  - Tal estratégia permite resolver recursivamente os subproblemas, combinando as soluções encontrada para retornar uma solução do problema original.



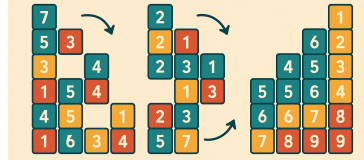
Algoritmo de Ordenação Merge Sort

- Divisão e conquista executa três passos a cada chamada recursiva:
  - Passo 1: dividir o problema original em uma certa quantidade de instâncias menores, os subproblemas..
  - Passo 2: conquistar os subproblemas, aplicando recursivamente sua divisão ou, obtendo a solução em instâncias reduzidas de tais subproblemas
  - Passo 3: combinar as soluções obtidas para os subproblemas para retornar a solução do problema original.



Algoritmo de Ordenação Merge Sort

- Merge sort utiliza os três passos do paradigma de divisão e conquista:
  - Passo 1: divide o problema de ordenação com n elementos em duas instâncias do subproblema original com n/2 elementos.
  - Passo 2: aplica recursividade para ordenar os subproblemas via intercalação
  - Passo 3: intercala as soluções ordenadas dos subproblemas retornando uma nova solução ordenada.



Divisão

p			r	p			r
1	2	3	4	5	6	7	8
50	20	40	70	10	30	20	60

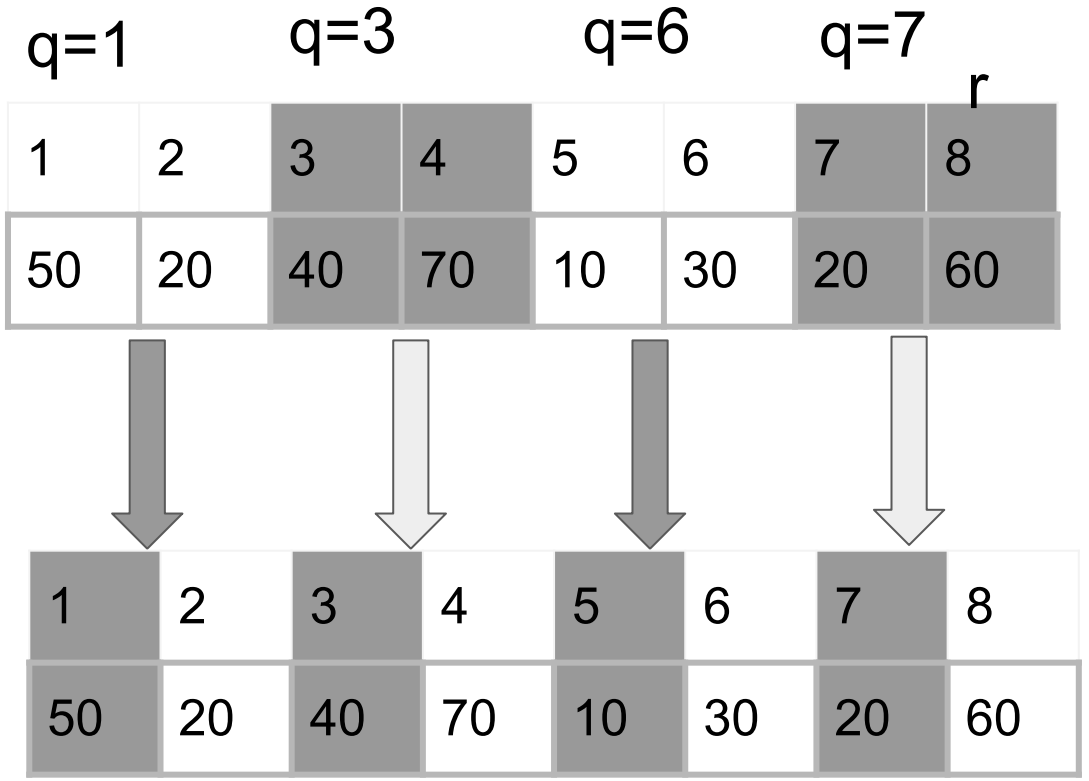
q=2 q=6

```
MERGE-SORT(A, p, r)
1  if p < r
2      q = ⌊(p + r)/2⌋
3      MERGE-SORT(A, p, q)
4      MERGE-SORT(A, q + 1, r)
5      MERGE(A, p, q, r)
```

p	r	p	r	p	r	p	r
1	2	3	4	5	6	7	8
50	20	40	70	10	30	20	60

Divisão

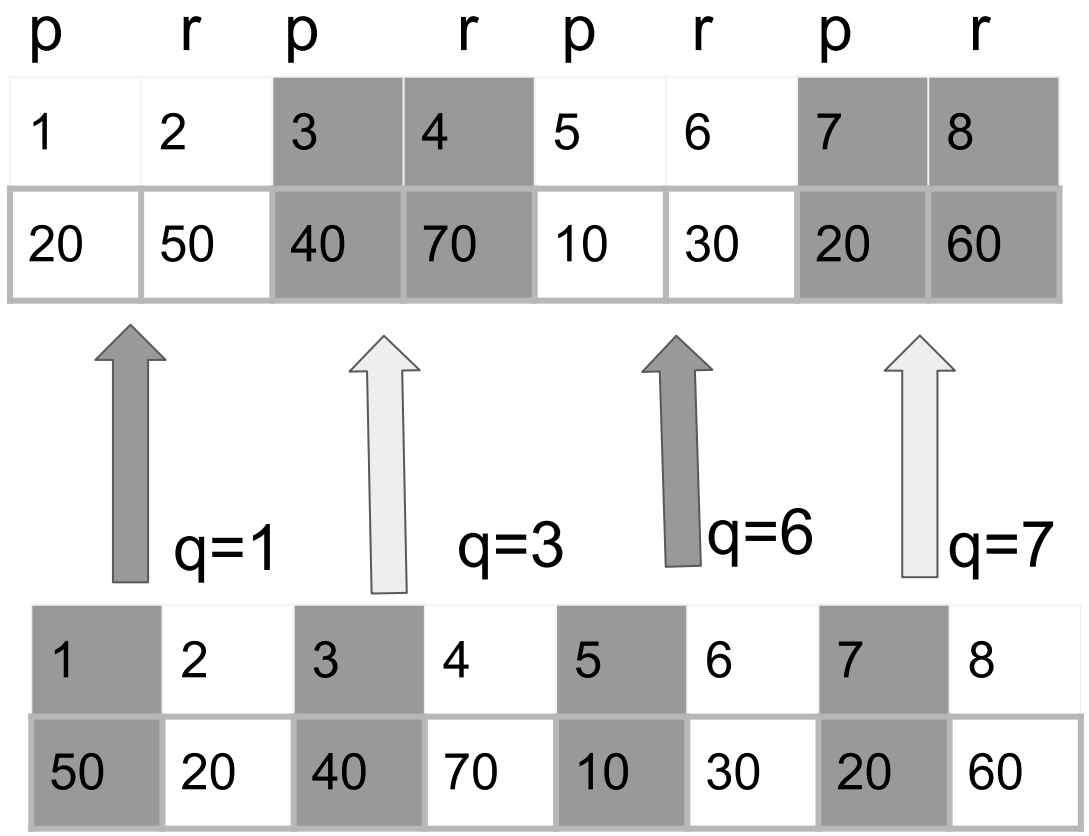
```
MERGE-SORT(A, p, r)
1  if p < r
2      q = ⌊(p + r)/2⌋
3      MERGE-SORT(A, p, q)
4      MERGE-SORT(A, q + 1, r)
5      MERGE(A, p, q, r)
```





Conquista

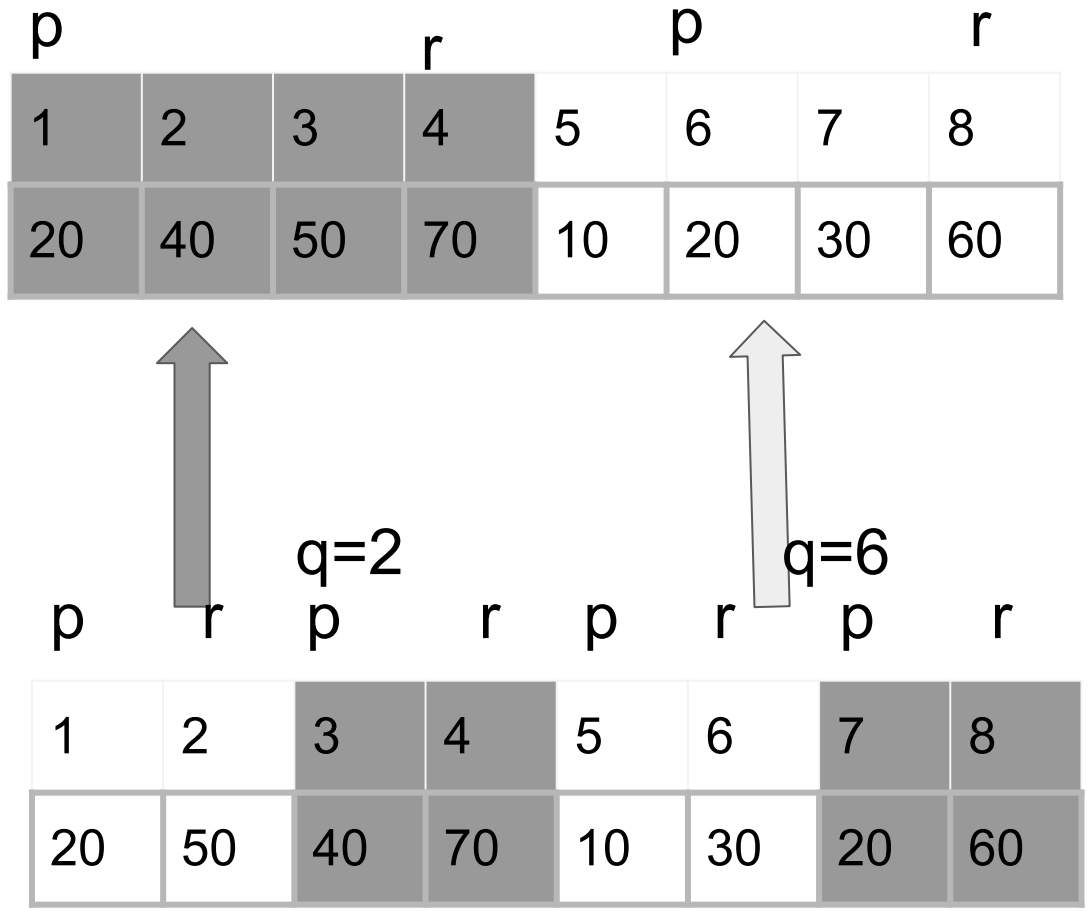
```
MERGE-SORT(A, p, r)
1  if p < r
2      q = ⌊(p + r)/2⌋
3      MERGE-SORT(A, p, q)
4      MERGE-SORT(A, q + 1, r)
5      MERGE(A, p, q, r)
```



r

Conquista

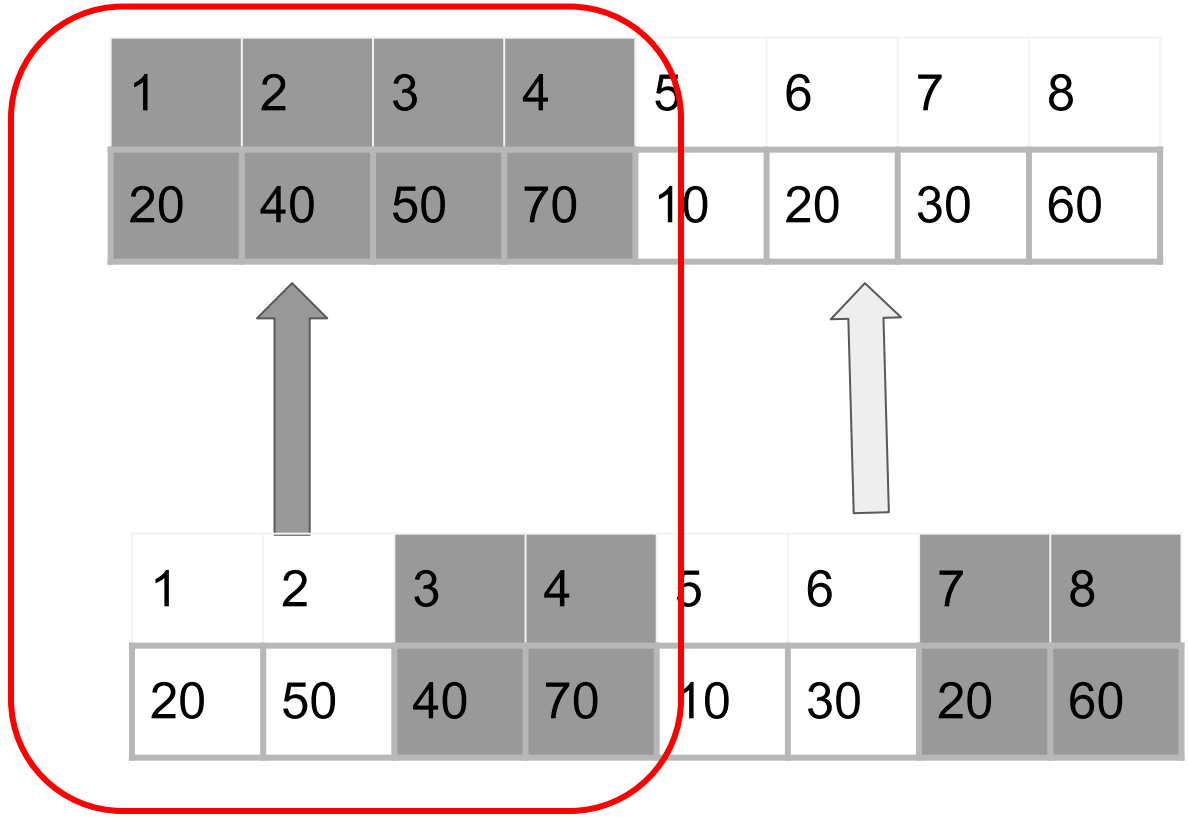
```
MERGE-SORT(A, p, r)
1  if p < r
2      q = ⌊(p + r)/2⌋
3      MERGE-SORT(A, p, q)
4      MERGE-SORT(A, q + 1, r)
5      MERGE(A, p, q, r)
```

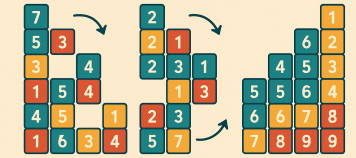


Conquista

```
MERGE(A, p, q, r)
1  n1 = q − p + 1
2  n2 = r − q
3  let L[1..n1 + 1] and R[1..n2 + 1] be new arrays
4  for i = 1 to n1
5      L[i] = A[p + i − 1]
6  for j = 1 to n2
7      R[j] = A[q + j]
8  L[n1 + 1] = ∞
9  R[n2 + 1] = ∞
10 i = 1
11 j = 1
12 for k = p to r
13     if L[i] ≤ R[j]
14         A[k] = L[i]
15         i = i + 1
16     else A[k] = R[j]
17         j = j + 1
```

p=1 q=2 r=4





r

```
MERGE-SORT(A, p, r)
1  if p < r
2        q = ⌊(p + r)/2⌋
3        MERGE-SORT(A, p, q)
4        MERGE-SORT(A, q + 1, r)
5        MERGE(A, p, q, r)
```

p r

1	2	3	4	5	6	7	8
10	20	20	30	40	50	60	70

q=4

p r p

1	2	3	4	5	6	7	8
20	40	50	70	10	20	30	60



Analisando a Complexidade do Merge Sort

- Na análise do MERGE-SORT, vamos supor que o tamanho do problema computacional de ordenação seja 2^n.
  - O algoritmo funciona corretamente para qualquer tamanho de instância do problema, seja o tamanho par ou ímpar.
  - A hipótese de tamanho como potência de 2 simplifica a análise da relação de recorrência do Merge Sort.



Analisando a Complexidade do Merge Sort

- Vamos estabelecer a relação de recorrência para o pior caso do tempo no tempo de execução T(n) durante a etapa de ordenação por intercalação do Merge Sort.
- Ordernar um único elemento (n=1) tem um custo de tempo computacional constante.
- Precisamos calcular o tempo de execução para a etapa recursiva quando n > 1, avaliando os passos de dividir, conquistar e combinar executados



Analisando a Complexidade do Merge Sort

- Passo 1 (Divisão):
  - Determina a divisão da instância do problema pelo seu ponto médio.
    - Esse cálculo tem um custo de tempo constante:

$$D(n) = \theta(1).$$

- Passo 2 (Conquista):
  - Deve-se obter a solução para dois subproblemas com tamanho n/2.
  - Essa etapa impacta em 2T(n/2) no tempo de execução.
- Passo 3 (Combinação): A rotina MERGE gasta um tempo θ(n), conforme veremos a seguir, assim:

$$C(n) = \theta(n).$$



Analisando a Complexidade do Merge Sort

```
MERGE-SORT(A, p, r)
1  if p < r
2      q = ⌊(p + r)/2⌋
3      MERGE-SORT(A, p, q)
4      MERGE-SORT(A, q + 1, r)
5      MERGE(A, p, q, r)
```

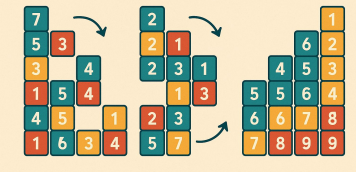
D(n): Tempo para dividir o problema

C(n): Tempo para combinar o problema

Relação de recorrência:

$T(1) = \Theta(1)$ n=1.

$T(n) = 2T(n/2)+D(n)+C(n)$ n>1.



Analisando a Complexidade do Merge Sort

```
MERGE(A, p, q, r)
1     n1 = q – p + 1
2     n2 = r – q
3     sejam L[1..n1 + 1] e R[1.. n2 + 1] novos arranjos
4     for i = 1 to n1
5        L[i] = A[p + i – 1]
6     for j = 1 to n2
7        R[j] = A[q + j]
8     L[n1 + 1] = ∞
9     R[n2 + 1] = ∞
10    i = 1
11    j = 1
12    for k = p to r
13       if L[i] ≤ R[j]
14          then A[k] = L[i]
15             i = i + 1
16          else A[k] = R[j]
17             j = j + 1
```

- Analisando a rotina Merge temos:
  - θ(1), tempo constante, nas linhas de 1 a 3 e 8 a 11.
  - θ(n1+n2)=θ(n) nas linha 4 a 7.
  - n iterações nas linhas 12 a 17
- Logo, a complexidade da etapa de combinação (Merge) será:

C(n) = θ(n)



Analisando a Complexidade do Merge Sort

```
MERGE-SORT(A, p, r)
1  if p < r
2      q = ⌊(p + r)/2⌋
3      MERGE-SORT(A, p, q)
4      MERGE-SORT(A, q + 1, r)
5      MERGE(A, p, q, r)
```

Relação de recorrência:

$T(n) = \Theta(1)$ n=1.

$T(n) = 2.T(n/2)+D(n) + C(n)$ n>1.

$$T(n) = \begin{cases} \Theta(1) & n=1 \\ 2T(n/2) + \Theta(n) & n>1 \end{cases}$$



Analisando a Complexidade do Merge Sort

$$T(n) = \begin{cases} \Theta(1) & n=1 \\ 2T(n/2) + \Theta(n) & n>1 \end{cases}$$

- Vamos aplicar o Método Mestre.
  - Temos a=2, b=2, e f(n)=Θ($n^{\log_b(a)}$)=Θ(n)
- Logo, podemos aplicar o caso 2 do teorema mestre.
  - Caso2: se f(n) = Θ($n^{\log_b(a)}$), então T(n) = Θ($n^{\log_b(a)}$ lg n)



Analisando a Complexidade do Merge Sort

$$T(n) = \begin{cases} \Theta(1) & n=1 \\ 2T(n/2) + \Theta(n) & n>1 \end{cases}$$

- Para a=2, b=2, e f(n)=Θ(n), temos

$$T(n) = \Theta(n^{\log_2(2)} \lg n)$$

$$= \Theta(n \lg n)$$



Analisando a Complexidade do Merge Sort

- Merge sort realiza o mesmo número de operações de comparação e atribuição para um array de um tamanho específico.
- Logo, apresenta a mesma complexidade O(n log n) de tempo no pior e melhor caso, assim como no caso médio.



Analisando a Complexidade do Merge Sort

- Merge sort, assim como bubble, insertion e selection sort, é um algoritmo baseado em comparações (comparison-based sorting algorithm).
- Merge sort também utiliza a estratégia de divisão e conquista no processo de ordenação.



Comparando Merge e Quick Sort

- Complexidade de espaço
    - Merge sort requer duas sublistas além da lista original, levando a criação de n elementos em memória e uma complexidade de espaço O(n).
- Merge sort tem um desempenho satisfatório em conjuntos de dados grandes ou pequenos.
- Merge sort é um algoritmo de classificação estável, ou seja, mantém a ordem relativa de dois elementos iguais.



Algoritmo de Ordenação Quick Sort

- Proposto por Charles Antony Richard Hoare, cientista da computação britânico, em 1960.
- Algoritmo de ordenação mais utilizado no mundo por conta da sua rapidez e eficiência.

```
Quicksort(L, p, r)
1.if (p < r)
2. q = Partition(L, p, r)
3. Quicksort(L, p, q)
4. Quicksort(L, q+1, r)
```



Algoritmo de Ordenação Quick Sort

- Executa a ordenação por comparação e troca dos itens
- Não demanda memória extra na ordenação.
- Executa a ordenação dentro da própria estrutura que armazena os valores (in-place algorithm).

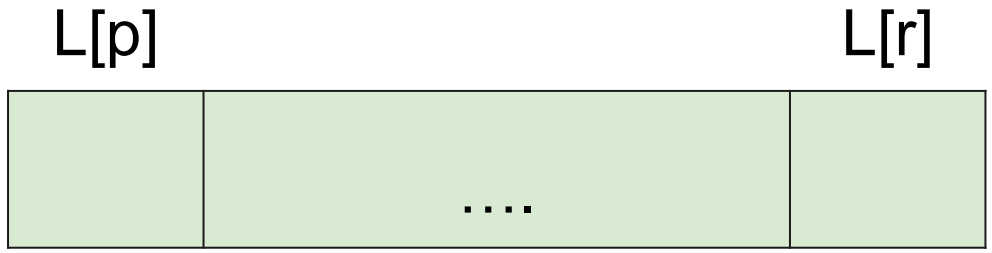
```
Quicksort(L, p, r)
1.if (p < r)
2. q = Partition(L, p, r)
3. Quicksort(L, p, q)
4. Quicksort(L, q+1, r)
```



Algoritmo de Ordenação Merge Sort

- Também utiliza o paradigma da divisão e conquista.
- Suponha dados de entrada armazenados em uma lista L[p...r].

```
Quicksort(L, p, r)
1.if (p < r)
2. q = Partition(L, p, r)
3. Quicksort(L, p, q)
4. Quicksort(L, q+1, r)
```

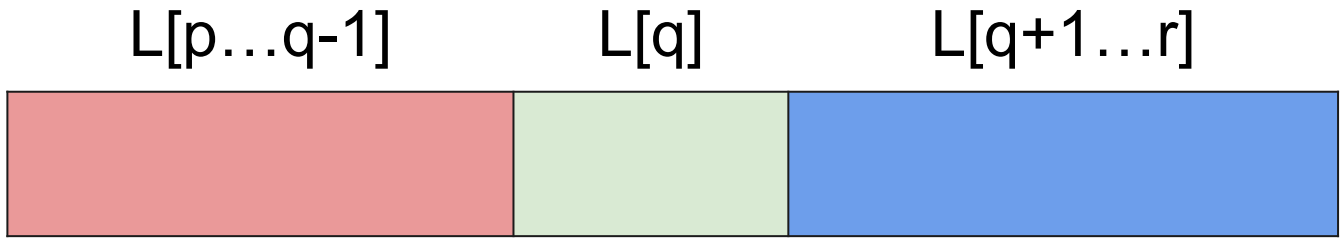




Algoritmo de Ordenação Merge Sort

- Divisão:
  - Particiona L[p .. r] em duas listas L[p .. q – 1] e L[q + 1 .. r]
  - Cada elemento de L[p .. q – 1] será menor ou igual a L[q]
  - Cada elemento de L[q+1...r] será maior ou igual a L[q].
  - L[q] é chamado pivô do particionamento.

```
Quicksort(L, p, r)
1.if (p < r)
2.  q = Partition(L, p, r)
3.  Quicksort(L, p, q)
4.  Quicksort(L, q+1, r)
```





Algoritmo de Ordenação Quick Sort

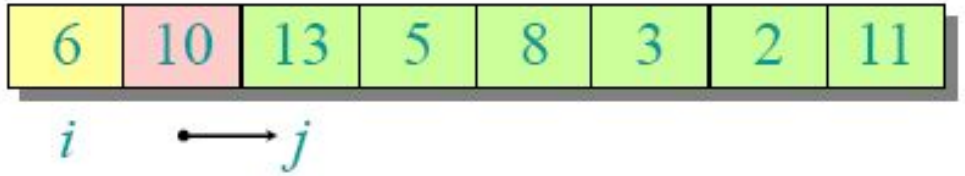
- Conquista:
  - Chamadas recursivas ao quicksort ordenam L[p .. q –1] e L[q + 1 .. r].
- Combinação:
  - Não se faz necessária já que a etapa de conquista leva ao ordenamento do vetor.

```
Quicksort(L, p, r)
1.if (p < r)
2. q = Partition(L, p, r)
3. Quicksort(L, p, q)
4. Quicksort(L, q+1, r)
```



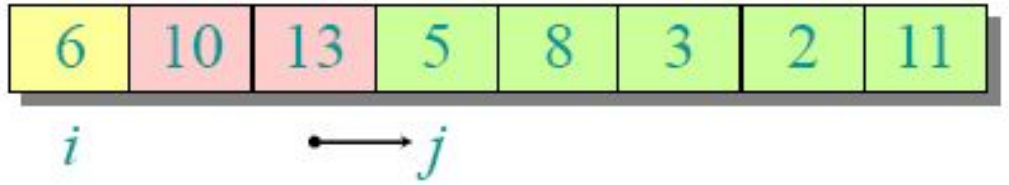
Algoritmo de Ordenação Quick Sort

```
Particao(L,p,r)//pivô=primeiro
1. valor = L[p]
2. i = p
3. Para j = p+1 até r faça
4.    Se L[j] ≤ valor
5.     i = i + 1
6.     troca(L[i],L[j])
7. troca(L[i],L[p])
8. retorna  i
```



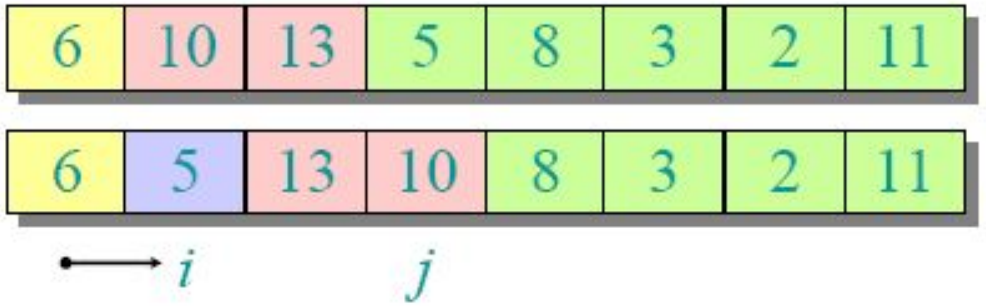
Algoritmo de Ordenação Quick Sort

```
Particao(L,p,r)//pivô=primeiro
1. valor = L[p]
2. i = p
3. Para j = p+1 até r faça
4.    Se L[j] ≤ valor
5.     i = i + 1
6.     troca(L[i],L[j])
7. troca(L[i],L[p])
8. retorna  i
```



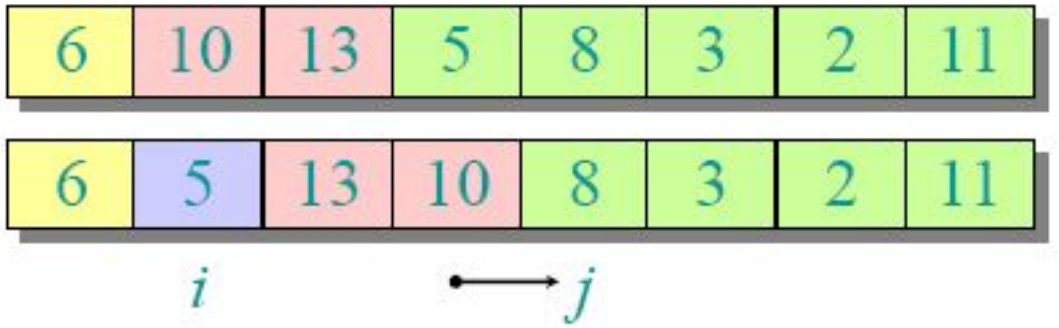
Algoritmo de Ordenação Quick Sort

```
Particao(L,p,r)//pivô=primeiro
1. valor = L[p]
2. i = p
3. Para j = p+1 até r faça
4.    Se L[j] ≤ valor
5.     i = i + 1
6.     troca(L[i],L[j])
7. troca(L[i],L[p])
8. retorna  i
```



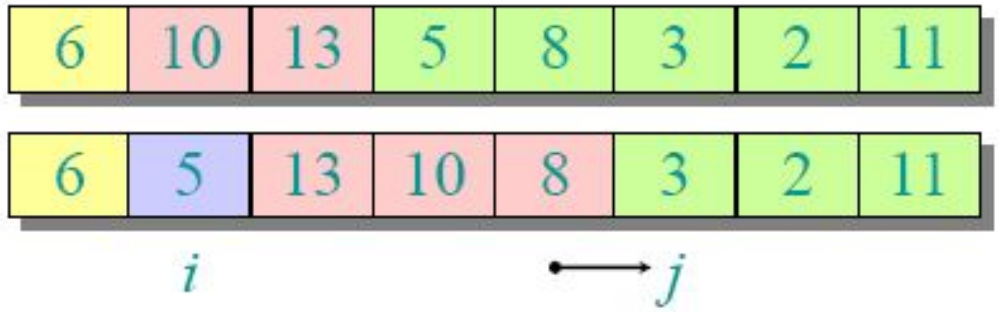
Algoritmo de Ordenação Quick Sort

```
Particao(L,p,r)//pivô=primeiro
1. valor = L[p]
2. i = p
3. Para j = p+1 até r faça
4.    Se L[j] ≤ valor
5.      i = i + 1
6.      troca(L[i],L[j])
7. troca(L[i],L[p])
8. retorna  i
```



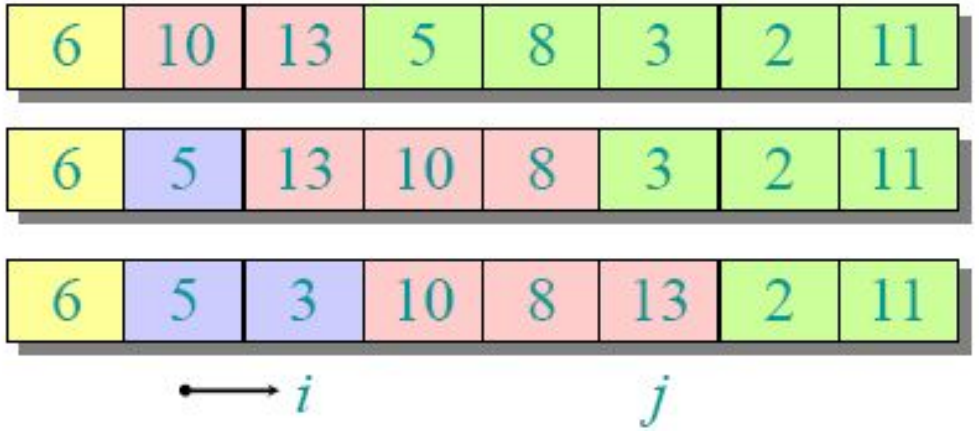
Algoritmo de Ordenação Quick Sort

```
Particao(L,p,r)//pivô=primeiro
1. valor = L[p]
2. i = p
3. Para j = p+1 até r faça
4.    Se L[j] ≤ valor
5.     i = i + 1
6.     troca(L[i],L[j])
7. troca(L[i],L[p])
8. retorna  i
```



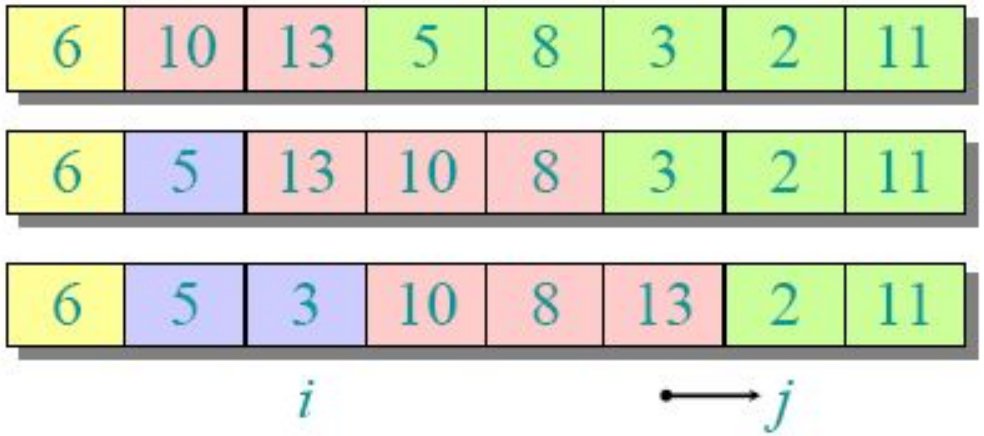
Algoritmo de Ordenação Quick Sort

```
Particao(L,p,r)//pivô=primeiro
1. valor = L[p]
2. i = p
3. Para j = p+1 até r faça
4.    Se L[j] ≤ valor
5.     i = i + 1
6.     troca(L[i],L[j])
7. troca(L[i],L[p])
8. retorna  i
```



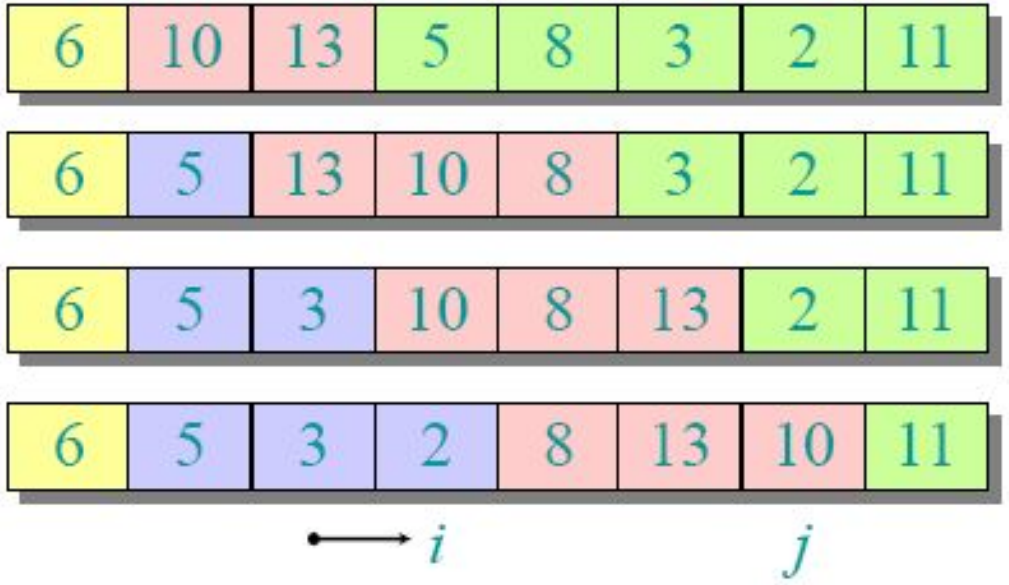
Algoritmo de Ordenação Quick Sort

```
Particao(L,p,r)//pivô=primeiro
1. valor = L[p]
2. i = p
3. Para j = p+1 até r faça
4.    Se L[j] ≤ valor
5.      i = i + 1
6.      troca(L[i],L[j])
7. troca(L[i],L[p])
8. retorna  i
```



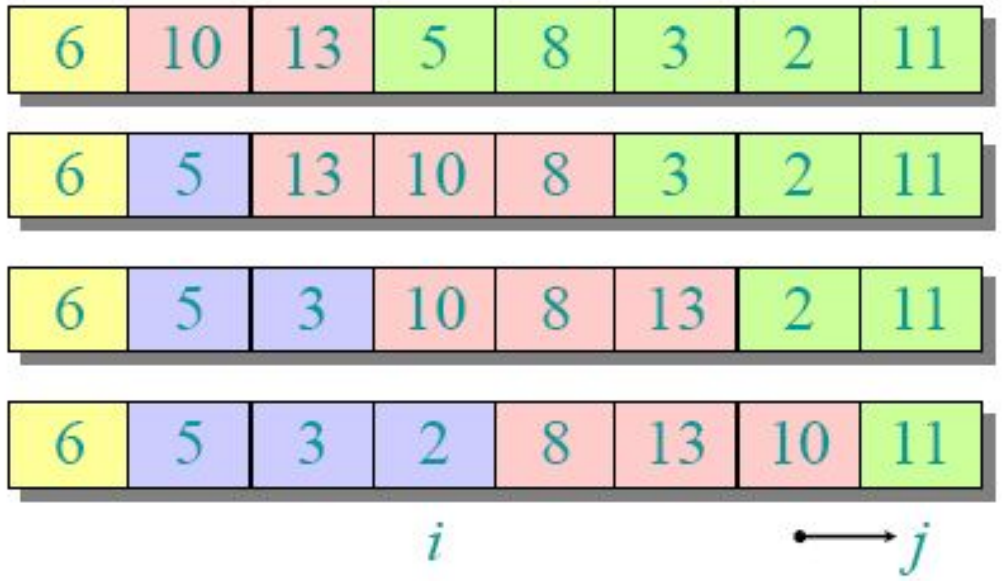
Algoritmo de Ordenação Quick Sort

```
Particao(L,p,r)//pivô=primeiro
1. valor = L[p]
2. i = p
3. Para j = p+1 até r faça
4.    Se L[j] ≤ valor
5.     i = i + 1
6.     troca(L[i],L[j])
7. troca(L[i],L[p])
8. retorna  i
```



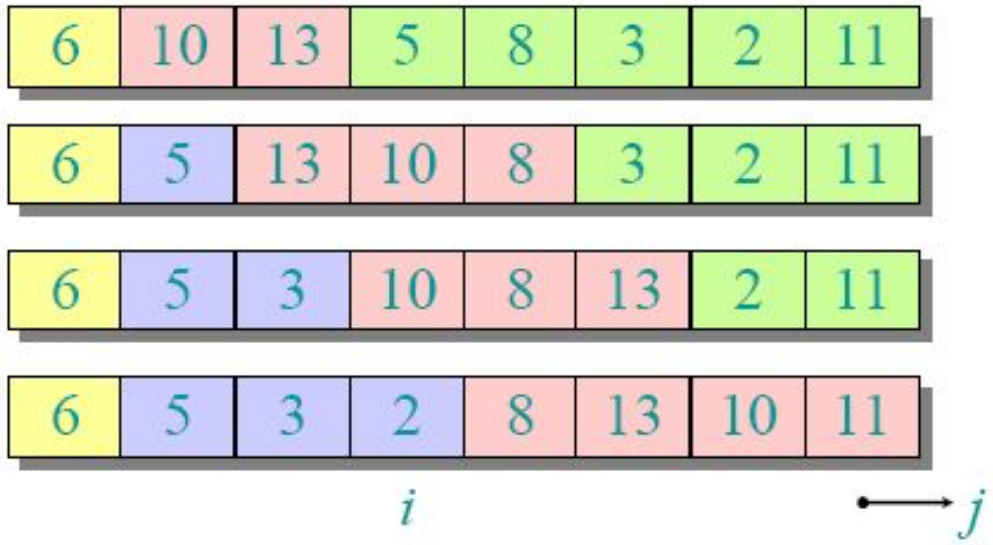
Algoritmo de Ordenação Quick Sort

```
Particao(L,p,r)//pivô=primeiro
1. valor = L[p]
2. i = p
3. Para j = p+1 até r faça
4.    Se L[j] ≤ valor
5.     i = i + 1
6.     troca(L[i],L[j])
7. troca(L[i],L[p])
8. retorna  i
```



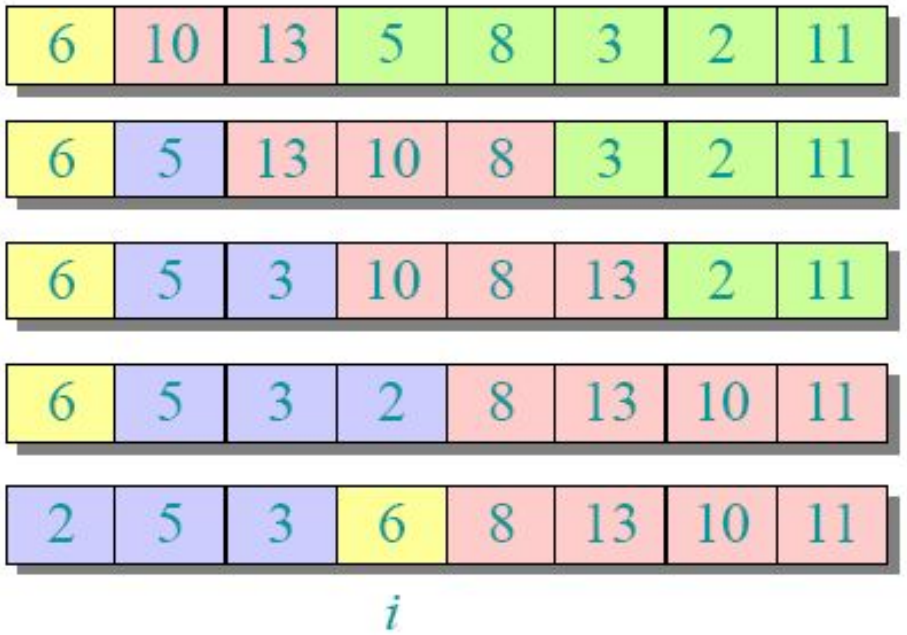
Algoritmo de Ordenação Quick Sort

```
Particao(L,p,r)//pivô=primeiro
1. valor = L[p]
2. i = p
3. Para j = p+1 até r faça
4.    Se L[j] ≤ valor
5.      i = i + 1
6.      troca(L[i],L[j])
7. troca(L[i],L[p])
8. retorna  i
```



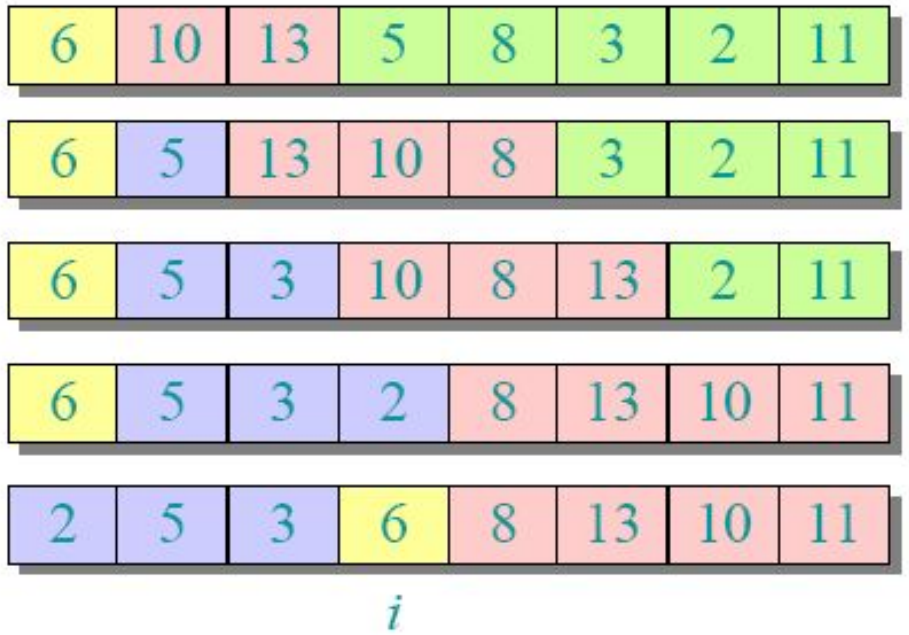
Algoritmo de Ordenação Quick Sort

```
Particao(L,p,r)//pivô=primeiro
1. valor = L[p]
2. i = p
3. Para j = p+1 até r faça
4.    Se L[j] ≤ valor
5.      i = i + 1
6.      troca(L[i],L[j])
7. troca(L[i],L[p])
8. retorna  i
```



Algoritmo de Ordenação Quick Sort

```
Quicksort(L, p, r)
1.if (p < r)
2. q = Partition(L, p, r)
3. Quicksort(L, p, q)
4. Quicksort(L, q+1, r)
```





Complexidade do Particionamento

- Pior caso ocorre quando o algoritmo de particionamento retorna um vetor com n-1 elementos e outro um único elemento.

```
Particao(L,p,r)//pivô=primeiro
1. valor = L[p]
2. i = p
3. Para j = p+1 até r faça
4.    Se L[j] ≤ valor
5.      i = i + 1
6.      troca(L[i],L[j])
7. troca(L[i],L[p])
8. retorna  i
```



Complexidade do Particionamento

- Suponha que o pior caso ocorra em toda chamada recursiva do Quick sort.
- O custo de particionamento é θ (n)

```
Particao(L,p,r)//pivô=primeiro
1. valor = L[p]
2. i = p
3. Para j = p+1 até r faça
4.    Se L[j] ≤ valor
5.      i = i + 1
6.      troca(L[i],L[j])
7. troca(L[i],L[p])
8. retorna  i
```



Complexidade do Quick Sort

- Assim, temos a relação de recorrência para o tempo de execução:

$$T(n)=T(n-1) + \theta(n)$$

```
Quicksort(L, p, r)
1.Se (p < r) então
2. q = Partition(L, p, r)
3. Quicksort(L, p, q)
4. Quicksort(L, q+1, r)
```



Complexidade do Quick Sort

- Expandindo a relação de recorrência, temos:

$T(n) = T(n-1) + \theta(n)$

$= T(n-1) + cn$

$= T(n-2)+c(n-1) + cn$

$=T(n-3)+c(n-2)+c(n-1)+cn$

$=\ldots=T(n-k)+c(n-(k-1))+c(n-(k-2))+\ldots+c(n-1)+cn$

```
Quicksort(L, p, r)
1.Se (p < r) então
2. q = Partition(L, p, r)
3. Quicksort(L, p, q)
4. Quicksort(L, q+1, r)
```



Complexidade do Quick Sort

- Expandindo a relação de recorrência, temos:

$T(n) = T(n-\mathbf{k}) + c(n-\mathbf{(k-1)}) + c(n-\mathbf{(k-2)}) + \ldots + c(n-\mathbf{1}) + cn$

Para k=n, temos.

$= T(0) + c.1 + c.2 + c.3 + \ldots + c.(n-1) + c.n =$

$= c(1+2+3+4\ldots+n-1+n) =$

$= c(n(n+1)/2) = c(n^2+n)/2$

$= O(n^2)$

```
Quicksort(L, p, r)
1.Se (p < r) então
2. q = Partition(L, p, r)
3. Quicksort(L, p, q)
4. Quicksort(L, q+1, r)
```



Complexidade do Quick Sort

- Logo, Quicksort é $O(n^2)$ no pior caso.

```
Quicksort(L, p, r)
1.Se (p < r) então
2. q = Partition(L, p, r)
3. Quicksort(L, p, q)
4. Quicksort(L, q+1, r)
```



Complexidade do Quick Sort

- O melhor caso ocorre quando há uma divisão equitativa.
  - Partition retorna dois subproblemas com tamanho $\lfloor n/2 \rfloor$ e $\lfloor n/2 \rfloor - 1$.
- Teremos a relação de recorrência abaixo para o tempo de execução do algoritmo:

$$T(n) = 2T(n/2) + \theta(n),$$

```
Quicksort(L, p, r)
1.Se (p < r) então
2. q = Partition(L, p, r)
3. Quicksort(L, p, q)
4. Quicksort(L, q+1, r)
```



Complexidade do Quick Sort

$$T(n) = 2T(n/2) + \theta(n),$$

- Aplicando o Teorema mestre temos $T(n)=aT(n/b)+f(n)$ com $f(n)=cn$, $a=2$ e $b=2$
- Caso2: se $f(n) = \Theta(n^{\log_b(a)})$, entao $T(n) = \Theta(n^{\log_b(a)} \lg n)$
  - $f(n)=cn=\Theta(n^{\log_2(2)})=\Theta(n)$, então $T(n)=\Theta(n \lg n)$
- Quicksort é $\Theta(n \lg n)$ no melhor caso.



Complexidade do Quick Sort

- No caso médio, quick sort apresenta um tempo de execução mais próximo do melhor caso que do pior caso.
- Por exemplo, caso o particionamento uma divisão proporcional de 9 para 1, aparentemente bastante desequilibrada, teremos a recorrência

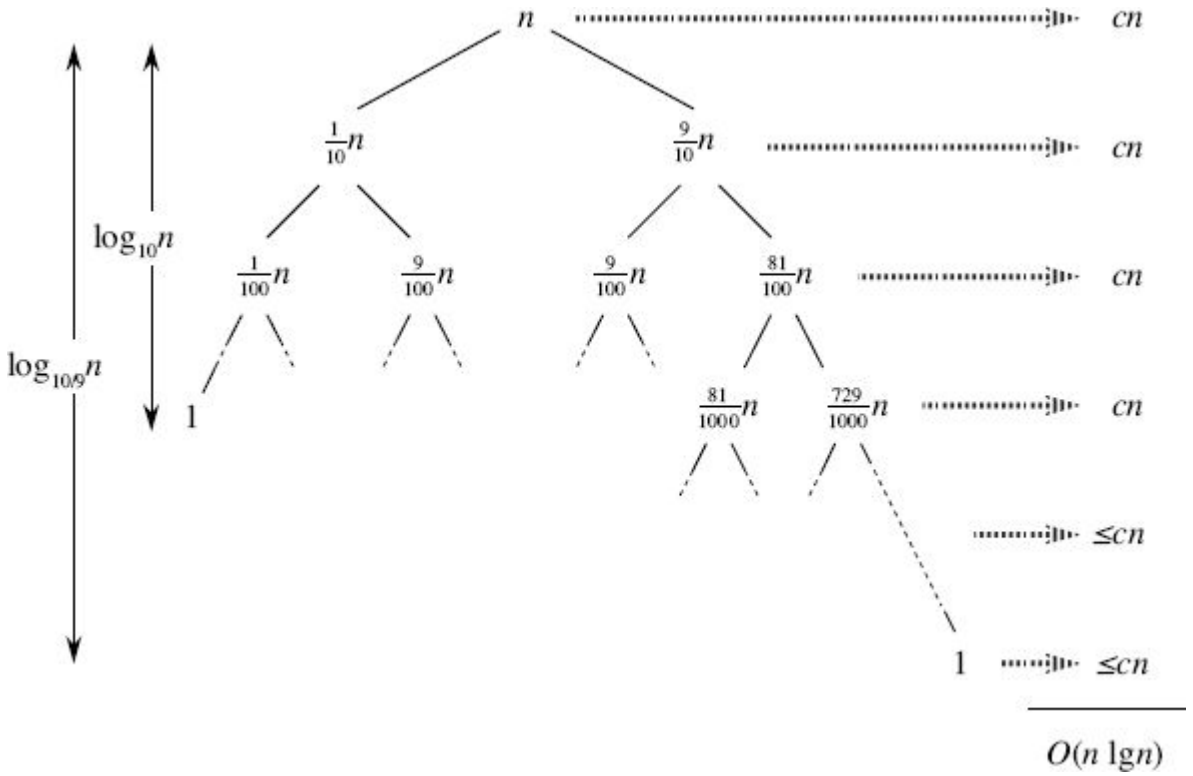
$$T(n) = T(9n/10) + T(n/10) + cn,$$



Complexidade do Quick Sort

- T(n) = T(9n/10) + T(n/10) + cn
  - Usando árvore de recursão vemos que T(n)=O(nlg n)
- Segundo Cormen,(2022), o algoritmo terá um tempo de execução O(nlogn) sempre que uma divisão retornada por partition tiver proporcionalidade constante.

Complexidade do Quick Sort



Fonte: Cormen, 2022

- Cada nível da árvore tem custo cn até atingir o passo base no nível log10 n.
- Nesse nível temos θ(lg n)
- Nos níveis seguinte, o custo fica limitado à cn.
- A recursão termina no nível log10/9 n = θ(lg n).
- Logo, temos um custo total O(n lg n), assintoticamente o mesmo custo se a divisão fosse ao meio.



Complexidade do Quick Sort

- Quick sort, assim como bubble, insertion e selection sort, é um algoritmo baseado em comparações (comparison-based sorting algorithm).
- Quick sort também utiliza a estratégia de divisão e conquista no processo de ordenação.



Comparando Merge e Quick Sort

- Merge e quick sort diferem na forma como tratam o particionamento e ordenação
  - Merge sort divide a lista em duas partições praticamente iguais, diferindo em 1 para tamanhos ímpares.
  - Merge sort conquista (junta) partições de forma ótima, formando uma nova partição ordenada.
  - Quick sort gera partições que podem ter tamanhos diferentes baseado na forma como o pivô é selecionado.



Comparando Merge e Quick Sort

- Quick sort executa parte da ordenação durante a etapa de divisão, enquanto merge sort faz isso durante a etapa de conquista ou merging.
- Quick sort se comporta melhor em conjuntos de dados distribuídos de forma uniforme ou conjunto de dados pequenos.



Comparando Merge e Quick Sort

- Quicksort é um in-place algorithm, ou seja, realiza classificação local com complexidade de memória é O(1).
- Quicksort é um algoritmo de ordenação instável, ou seja, pode alterar a ordem relativa de dois elementos iguais.